QR code decoder (mkQRDecoder): design and verification report

CS6230 CAD for VLSI, 2026. Physical-design results are in pd_report.pdf; this report covers the architecture, how it meets the specification, and how it was verified.

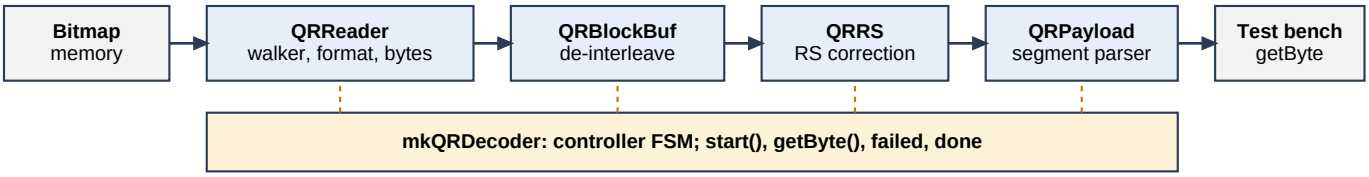
Roll number: EE23B056
Team: EE23B056 Pozhilan Karthikeyan; EE23B081 P Tejasvin

0 / 1,614	16 / 16	1.00 ns	71.90%
whole-decoder mismatches	planted bugs caught	f2f slack 0.00 after place_opt	post-PlaceOpt utilization

1. Summary

- **Function.** The decoder implements G1 to G4 and C1 to C5 of the specification. In our whole-decoder regression of 1,614 symbols (1,168 that must decode and 446 that must fail, run back to back without reset, with random stalls on getByte) there are 0 mismatches against the Python reference model.
- **Architecture.** Small and serial: one module read per cycle, one byte-wide pipeline, one shared general GF(2^8) multiplier and one inverter in the Reed-Solomon stage. A 1,024-byte block buffer holds as many blocks as fit; larger symbols are read again for the next group of blocks (area traded for cycles).
- **Physical design.** Four synthesis designs, each differing from the previous in one BSV file, took the clock from 2.7 ns to 1.0 ns with no timing violations (Section 6). The best design places at 71.90% utilization in a 170 x 170 um core, with the f2f path meeting 1.00 ns.

2. Architecture



The top level (QRDecoder.bsv) is one controller rule that sequences the units; the data path between units is by method calls, so each unit owns its own registers. The top-level states are: TIdle, TFmt (read the format information), TCfg (look up the block structure for version and level), TPlan/TWaitPlan (choose the group of blocks for this pass), TFill (read the whole symbol and forward every codeword byte to the buffer), TRewind, TBlk/TRun/TAdv (correct one block and forward its data bytes to the payload parser), TFinal (wait for the parser to finish) and TBad (bad side length). Reading and Reed-Solomon never overlap, so the reader is always idle when a failure is noticed, and any failure simply returns the controller to TIdle: the next start() reconfigures every unit.

2.1 Units

Unit (file)	What it does	Design decisions
Walker (QRWalker.bsv)	Visits the data modules in the zigzag order of Section 1.3; isFunctionModule and maskInverts are pure functions of (row, col, version).	Function modules are stepped over without being handed out; the walker knows nothing of memory or bytes. The 8 mask conditions and the alignment-pattern map are computed combinationally from the QRTables parameters.
Format (QRFormat.bsv)	Table 1 positions; nearest valid word of the 32 for each copy; picks the nearer copy (copy 1 on a tie); fails above distance 3 (G2).	Because valid words differ in at least 7 bits, at most one can be within 3 bits of a read word, so the nearest-word search is 32 independent "within 3 bits" tests whose results are ORed (this replaced a 32-deep compare-and-select chain, iteration 2).
Reader (QRReader.bsv)	Reads the 30 format modules, then every data module (one request per cycle), unmasks, packs 8 bits (first bit = MSB) into a byte and hands bytes out in reading order.	Follows C4: an answer arrives the cycle after the request and then holds, so the reader remembers that it asked and uses resp only in the following cycle. A one-byte output slot gives back-pressure; the 3 to 7 left-over modules of some versions are dropped.
Block buffer (QRBlocks.bsv)	De-interleaves into blocks (formula of Section 1.4) with counters only, no division, and stores a group of blocks back to back.	CAP = 1,024 bytes (compile-time knob). A pass keeps blocks g0..g1-1 (greedy); if blocks remain the top level reads the symbol again. Larger CAP means fewer passes but more area. Supports blocks of two lengths (n1, n2) and check bytes after the data.
Reed-Solomon (QRRS.bsv)	Per block: syndromes, Berlekamp-Massey, evaluator, Chien search with Forney values, write-back, re-check of syndromes, then output of the data bytes.	One shared general multiplier and one inverter, everything else byte-serial constant multipliers (cycles traded for area). Fails if the locator degree exceeds e/2, if the number of roots differs from the degree, or if the syndromes are not zero after correction, so a damaged block is never silently miscorrected (A5).
GF(2^8) (QRGF.bsv)	Multiply, multiply by alpha, inverse (polynomial 0x11D, alpha = 2).	The inverse is a 256-entry table generated at compile time from the a^254 chain (kept as gfInvChain), so the values are identical by construction; this removed the critical path in iteration 3.
Payload (QRPayload.bsv)	Parses mode, count and characters for numeric, alphanumeric and byte modes (count widths by version class); emits one ASCII byte per character.	Bit accumulator plus a one-character hold-back so that last can be set on the final character; after the terminator or a failure further bytes are accepted and discarded so the producer never stalls. All G4 failures (unknown mode, segment past the end, group out of range) raise failed.

2.2 Interface contract

Rule	How it is met
G1 version	start(size) accepts 21 to 177 with (size - 17) divisible by 4, otherwise the controller goes to TBad and failed rises two cycles later (it stays low the cycle after start, as C2 requires).
G2 format	Both copies are read (30 requests); the nearer valid word is chosen, copy 1 on a tie; failure beyond distance 3.
G3 correction	Per-block Reed-Solomon; any block that cannot be corrected raises failed and returns to TIdle.
G4 payload	QRPayload with all failure cases; a failure found late fails the whole symbol.
C1 / C3 / C5	getByte has an implicit condition (a character is ready and failed is low) and start has an implicit condition (state is TIdle); neither carries always_ready or always_enabled. The test-bench build elaborates mkQRDecoder to Verilog and checks that RDY_start and RDY_getByte exist.
C2	failed and done are cleared in the cycle after start; exactly one rises at the end; done rises only after the parser has handed out every character (at once for an empty message).
C4	mem.req is issued only when the decoder wants a module; the reader trusts mem.resp only in the cycle after it asked.
A4 / A5	Function modules are never trusted: only the format modules and the data modules are read; damaged finder, timing or alignment modules cannot cause failure.

2.3 Choices the specification leaves open

These are shared by the reference model and the RTL, so a wrong reading would pass the comparison; they are listed so that they can be checked against the course forum. (1) A stream with fewer than 4 bits left and no terminator ends the message successfully. (2) A segment with count 0 is legal and decoding continues. (3) Numeric groups keep leading zeros (007 gives three bytes). (4) Format copy choice: copy 1 if its distance is not larger than copy 2, then fail if the chosen distance is above 3. (5) Any mode indicator other than 0001, 0010, 0100 and 0000 is unknown and fails (so ECI, Kanji and structured append fail). (6) A failure found late, for example a malformed segment after good ones, fails the whole symbol.

3. Performance model

The reader makes about one step per module visited, function modules included, so one pass over a symbol of side N costs roughly N^2 cycles; Reed-Solomon work per block is added on top. A symbol whose codewords do not fit the buffer is read once per group of blocks. In our regression, the cases run without back-pressure had a maximum latency of 37,388 cycles (start to the byte with last taken; slowest symbol: a full version 27 L message of 1,465 bytes) and a mean of 11,278 cycles. None of the four design iterations changed the number of cycles, so latency is identical across them. Latency is measured as in the specification, from the cycle in which start fires to the cycle in which the byte with last is taken; our own test bench allowed 3,000,000 cycles per symbol before declaring a hang.

4. Verification

Approach. A Python reference decoder (qr_model/) was written from the specification before the RTL; its tables are parsed from the supplied QRTables.bsv, and an independent encoder (checked bit for bit against segno on all 160 version/level pairs) builds the test symbols. Each hardware unit has its own test bench (Bluesim) and checker in verification/modules/, and the whole decoder is tested end to end. Everything runs from one command: sh run_all.sh (quick) or FULL=1 sh run_all.sh (all 1,614 symbols at the delivered 1,024-byte buffer and the planted-bug runs). verification/README.md says how to run it.

Unit	Checked against	Test set size / result
Model self-tests	spec definitions, segno	16 pytest tests: all 65,536 GF products, all 2^15 format words, all 160 version/level tables, segno round trip, all masks and modes, exactly t errors corrected and t+1 never silently accepted, the course sample symbol
walker	model (layout.py)	module order and function-module skipping for all 40 versions and all 8 masks
format	model (formatinfo.py)	32,768 of 32,768 words; 1,200 of 1,200 positions (40 versions x 2 copies x 15 bits); 400,000 of 400,000 random copy pairs identical
gf	spec definition and model	all 65,536 products; all 256 inverses; constants
blocks	interleaving formula of the spec	all 160 version/level pairs at several buffer sizes
reader	model	whole symbols through the memory port, format failures, bad sizes
rs	independent encoder and model decoder	4,099 of 4,099 blocks match: 1,214 clean, 1,727 corrected, 518 corrected with exactly t errors, 298 too damaged (must fail); includes blocks built to lie within reach of another codeword
payload	independent parser from the spec and model	2,014 cases (895 decode, 1,119 fail), 20,021 characters, 0 mismatches
top (whole decoder)	model, end to end	1,614 symbols (1,168 decode, 446 must fail), 1,024-byte buffer, back to back without reset, random getByte stalls: 0 mismatches

Whole-decoder test set. It covers every version 1 to 40 at every level, all eight masks and all three modes, multi-segment messages, the empty message, short final groups (one and two digits, one alphanumeric character), both sides of every count-width boundary, exactly floor(e/2) damaged bytes in every block of every version and level, a random number of damaged bytes per block, damaged check bytes, damaged function modules (finders, timing, alignment, version information), up to 3 wrong bits in one or both format copies, and the symbols that must fail: one damaged byte too many in a block, format information past repair, a bad side length and malformed payloads.

Planted bugs. To check that the tests can detect faults, single bugs were planted in a scratch copy of the RTL and the tests rerun. Top level, 9 of 9 caught: format level bits inverted; mask forgotten; next pass restarting at block 0; Reed-Solomon failure ignored; endOfData never sent; getByte still ready after failed; wrong check-byte count; done raised without waiting for the parser; side length never checked. Payload parser, 7 of 7 caught: count-width class boundary 26 to 27; numeric limit 999 to 1,023; alphanumeric pair limit 2,024 to 2,025; last flag never set; fewer than 4 bits left fails; bytes after the end not discarded; byte-mode count width 16 to 8.

What this does not cover. All tests run in Bluesim; Verilator, cocotb and code-coverage tools were not used, so no code-coverage figure is claimed. The model and the RTL share one reading of the open points in Section 2.3. The synthesized Verilog of designs 3 and 4 was checked to be identical (apart from the compiler timestamp line) to the Verilog generated by the tested build.

5. Design iterations (post-synthesis)

Each iteration read the worst timing paths, changed one BSV file, re-ran the regression (0 mismatches each time) and synthesized again. The full tables, the same-period comparison and the run folders are in pd_report.pdf.

#	Limit found in the previous report	Change (one file)	Result
1	Baseline: at 2.5 ns the worst paths start at the second format copy and end at the mask and level registers (32-deep compare-and-select chain)	none	meets 2.7 ns
2	same	QRFormat: 32 parallel within-3-bits tests	meets 2.1 ns
3	GF inverse built as a^254 from six multipliers in series (about 40 XOR levels)	QRGF: 256-entry compile-time table	meets 1.0 ns; at 1.9 ns cell area -5.9%
4	subtract and add in front of the 1,000-way buffer read decode	QRRS: registered buffer address (rdAddr), no extra state	meets 1.0 ns; at 1.0 ns cell area -7.7% vs design 3

6. Physical design summary

Stage	Clock (ns)	f2f slack (ns)	Leaf cells	Seq. cells	Power (mW)	Area (um^2)
DC, design 1	2.7	0.00	42,275	9,496	6.145	21,311 cell / 64,013 total
DC, design 2	2.1	0.00	40,862	9,496	7.780	20,382 cell / 62,688 total
DC, design 3	1.0	0.00	41,830	9,496	15.156	26,896 cell / 64,196 total
DC, design 4 (best)	1.0	0.00	40,408	9,508	15.653	24,828 cell / 61,137 total
ICC place_opt, design 4	1.0	0.00	38,074	9,508	14.710	19,724 cell; 169.98 x 169.23 um; 71.90% utilization

Synthesis is Design Compiler, non-topographical, with input and output delays of 40% of the period; placement is IC Compiler place_opt only (no CTS or routing). Total area from DC includes wire-load net area and is not comparable with the cell-only area from ICC. Details, definitions and limits are in pd_report.pdf.

7. Known limits

- Latency is not minimal: reading is one module per step including function modules, and a symbol larger than the 1,024-byte buffer is read several times. The buffer size is the knob (CAP) that trades area against cycles.
- Post-placement timing uses an ideal clock and estimated parasitics; it does not include clock-tree skew or routing.